

Agentic Computational Reproducibility of Published Research on CORE-Bench

CSE598 (Agentic AI) — Capstone Project Proposal

Student name	Jason Lunder
Project title	Agentic Computational Reproducibility of Published Research on CORE-Bench
Repository / notebook link	https://github.com/jlunder00/repro-agent
Configuration location	examples/ and .env.example

1 Problem Definition

Published code capsules often fail to run after release because of dependency rot, undocumented environments, or ambiguous run instructions. **User:** a researcher, reviewer, or student who holds a paper and its capsule, must confirm that the reported numbers come out of the code, and has no working environment for it. **Input:** a code capsule (source plus README and/or Dockerfile) and questions about its reported results; some questions ask about values shown in the capsule’s figures. **Output:** a `report.json` mapping each question to an answer. **Success:** every answer falls inside the benchmark’s 95% prediction interval built from three ground-truth runs (exact match when those runs agree). **Failure:** any wrong answer, a crash, or no file produced.

2 Motivation and Project Scope

We target CORE-Bench [Siegel et al., 2024b,a]: 270 tasks from 90 papers (Python and R), 45 train / 45 test papers, each at three tiers: *easy* (a successful run’s output is given, execution forbidden), *medium* (Dockerfile plus README), and *hard* (README only; the agent installs dependencies and infers commands).

Why agentic. The hard tier requires acting on an environment and observing state that is unavailable up front: tracebacks, file layout, version incompatibilities. The §4 trace shows two failures, the second visible only after the first fix; a single call cannot see them and a rule-based installer cannot read them. Structurally this is SWE-bench for environments: the agent changes the environment, not the code, and the target is a number, not a passing test.

Claim. CORE-Agent is AutoGPT in continuous mode with a \$4 budget; it already observes failures and re-plans, so “add an iterative loop” is not a contribution. The capstone adds two mechanisms CORE-Agent lacks: (a) *date-aware dependency resolution*: each `pip install` is resolved to the newest release predating the capsule’s publication date (available via `capsule_doi`), which fixes the `xlrd==1.2.0` class of failure before the traceback rather than after retries; (b) *base-image selection as an agent action* instead of a fixed per-language lookup. Ablation: the same loop with and without these components, at equal dollar cost.

Scope IN: all 90 capsules (45 train, 45 test), Python and R, at the easy and hard tiers; written and figure questions, with figures passed to the model as images. GPU-flagged capsules are run on CPU. **Scope OUT:** the medium tier (needs Docker-in-Docker), GPU execution, and repeated runs per task. **Feasibility:** the sandbox, grader, and single-pass pipeline exist, and one full sweep of 90 capsules at both tiers cost \$2.66 and 41 minutes of wall time; the resolver is one PyPI lookup per package and image selection is one sandbox parameter.

3 Runnable Baseline

A single-pass, no-retry pipeline using `claude-sonnet-5` through `litellm` and Docker: (1) fetch the capsule, (2) read the README, (3) one LLM call proposing install and run commands, (4) execute once in a base image chosen by capsule language (`python:3.11-slim` for Python, `r-base:4.4.1` for R; a fixed lookup, not a decision), (5) one LLM call extracting answers from the run output, the result files, and up to six of the capsule’s figures attached as images, (6) write `report.json`, (7) score with the vendored official grader. Each stage records success or failure, so a failed task is attributed to plan, execute, or extract.

Files: `run_baseline.py` (CLI); `src/repro_agent/dataset.py` (task list, filters), `capsules.py` (download, cache, tier preparation), `llm.py` (litellm wrapper, image input, cost tracking), `sandbox.py` (Docker execution), `baseline.py` (pipeline), `scoring.py` (vendored official grader).

Why a reasonable start: it is the floor. The experimental control for the capstone is the capstone loop at one iteration, sharing prompts and environment with the treatment arm, so the ablation isolates the two mechanisms rather than two codebases.

4 Test Case and Baseline Output

Example: `capsule-9052293` (1.0 MB), an ANP-TOPSIS tourism model. **Input:** the capsule plus “Report the closeness coefficient for location L1.” **Expected output:** `report.json` containing 0.844703753651819; all three ground-truth runs are identical, so an exact match is required.

```

$ python run_baseline.py --subset all --limit 45 --split test --tier easy
Task accuracy (easy): 22/45 = 48.9%
Total cost: $0.7945   Total time: 208.6s
Results written to results/baseline_easy_test_all.json

$ python run_baseline.py --subset all --limit 45 --split test --tier hard
Task accuracy (hard): 0/45 = 0.0%
Total cost: $0.4837   Total time: 724.2s
Results written to results/baseline_hard_test_all.json

```

Figure 1: Baseline output on the test split at both tiers. The train split gives 21/45 easy and 0/45 hard.

Easy (90 capsules, figures attached): 43/90 tasks (47.8%); 63/98 written questions (64.3%), 38/83 vision questions (45.8%), 101/181 overall (55.8%); 152 images sent; \$1.61 total, \$0.0178 per task. By split and language: test Python 15/22, test R 7/23, train Python 17/27, train R 4/18. Without images the model scored 7/54 on vision questions. **Hard (90 capsules):** 0/90 tasks, 0/181 questions; the execute stage succeeded 0/90 in every split and language; \$1.04 total, 22.0 s per task. No run was aborted: extraction ran on the failed output and returned null. For capsule-9052293 the plan was `pip install openpyxl pandas && cd code && python script.py`. Manual tracing shows two sequential fixes, the second visible only after the first: (i) `ModuleNotFoundError: xlrld`; (ii) `xlrld 2.x` dropped `.xlsx` support, so `xlrld==1.2.0` is required — an older release of the package just installed. With both, the capsule reproduces 0.844703753651819 exactly.

5 Reproducibility and Run Instructions

Python 3.10+. `pip install -r requirements.txt` (litellm is pinned <1.60; later releases fail on Python 3.10). Copy `.env.example` to `.env`, set `ANTHROPIC_API_KEY` or `OPENAI_API_KEY`, then `export $(grep -v '^#' .env | xargs)`. Task metadata and prompts are in `examples/`. Single capsule, easy: `python run_baseline.py --capsule capsule-9052293 --tier easy`. Full split: `python run_baseline.py --subset all --limit 45 --split test --tier easy`; repeat with `--split train` and `--tier hard`. The hard tier needs a running Docker daemon. Capsules are downloaded into `capsules/` on first use; output goes to `results/baseline_<tier>_<split>.json` (override with `--out`); the committed runs are `results/baseline_<tier>_<split>_all.json`. `--list` prints the selection without a key. Medium tier: not implemented.

6 Initial Evaluation Plan

Development uses the train split (45 tasks per tier); evaluation uses the test split (45 tasks per tier). Primary metric: official task accuracy. Secondary metrics: written-question and vision-question accuracy, since task accuracy is all-or-nothing. Also recorded: execution success rate, cost, latency, failure stage. Each arm runs at least 3 times per task because the model is sampled without a fixed temperature; results are mean and range at matched cost. Improvement means the arm with the resolver and image selection beats the arm without them at equal cost, same model, task set, and grader. The published numbers below are context only: different model, year, and harness; they are not a like-for-like baseline.

Agent	Model	Easy	Medium	Hard
CORE-Agent	GPT-4o	60.00%	57.78%	21.48%
AutoGPT	GPT-4o	35.56%	37.78%	6.67%
This baseline (test split, 45 tasks)	claude-sonnet-5	48.9%	—	0.0%

Rows 1–2: Siegel et al. [2024b], Table 5 (pass@1, test split, 45 tasks/tier). Row 3: §4, one run, not comparable.

7 Limitations and Next Steps

Figure cap: at most 6 figures are attached per capsule, chosen by sorted filename; 13 capsules have more, so a question about a figure outside that set stays unanswered. *Result truncation:* result files are concatenated in directory order up to a character budget, so an answer late in a long log can be cut off. *Single run:* one run per task with no fixed temperature, so repeated runs are needed before any comparison of arms. *All-or-nothing scoring:* per-question accuracy is reported alongside task accuracy. *Base image:* the per-language lookup is fixed; R hard-tier execution succeeded 0/41 even with the correct interpreter, so image choice must become a recorded variable in both arms. **Needed:** API and container budget for repeated sweeps (90 tasks × 2 arms × 3 runs, up to 900s each) and a Docker host. **Next:** the loop; the date-aware resolver; image selection; a verification run for extracted values; repeated runs.

References

- Zachary S. Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebel, and Arvind Narayanan. CORE-Bench GitHub repository. <https://github.com/siegelz/core-bench>, 2024a. Accessed 2026-09-06.
- Zachary S. Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebel, and Arvind Narayanan. CORE-Bench: Fostering the credibility of published research through a computational reproducibility agent benchmark. *arXiv preprint arXiv:2409.11363*, 2024b. URL <https://arxiv.org/abs/2409.11363>.